DS295R Agentic AI: Course Roadmap

Source: James Beeson, Student Course Lead (DS295R), voice-to-text roadmap, October 9, 2026
Prepared for: Andrew Seaman, October 11, 2026

The course moves you through four roles. Each one takes the agent further from “my agent, for me” toward “an agent other people depend on,” and each one needs the skills from the stage before it.

> **You → a company’s broken agent → a nonprofit with no agent → the course itself**

James’s roadmap is the source of truth here. Unit 1 details come from the course website, which James says is still being cleaned up, so they are labeled. Quotes from James are lightly cleaned from voice-to-text.

How to read this roadmap

1. **First pass: read the table below and stop.** Don’t look anything up. The goal is the shape, not the details.
2. **Before each class: find your place.** Ask which unit you’re in, what role you’re playing, and what question the unit is answering.
3. **After each class: file it.** Write one line: “Today I learned ___, which helps me build / diagnose / design an agent.”
4. **When a detail starts to bury you:** ask whether it changes the unit’s one question. If it doesn’t, park it on a “later” list and keep going.

The whole course in one table

Stage	Your role	The agent is for	The question it answers	What you produce
Unit 1: Build	Yourself	You	What is an agent?	A personal assistant agent (Project 1)
Unit 2: Diagnose	New hire at a company whose agent is broken	The company	What’s underneath an agent, and why does it break?	A diagnosis, then a fix
Bridge: Intent engineering	—	—	What is the agent actually for?	Not specified yet
Unit 3: Build for others	New hire at a nonprofit with no agent	People who aren’t tech-savvy	What should we build, and for whom?	A purpose-built agent, from scratch
Capstone (if time allows)	Contributor	The course and future students	What can I give back?	A fix or addition to the course

Read down the “agent is for” column: you, then a company, then people who aren’t tech-savvy, then future students. As the agent moves further from you, each stage adds a skill: building, then diagnosing, then working out what people actually need.

Unit 1: Build an agent for yourself

- **Your role:** yourself. James: you're "in the shoes of yourself," building "an agent for you."
- **The question:** what is an agent? James: "This is what an agent is."
- **What you learn:** the build concepts. On the website they come in this order:
  1. Call a model through an API key (Lesson 1.1)
  2. Write your own tools (Lesson 1.2)
  3. Add prebuilt tools and local files (Lesson 1.3)
  4. Move the agent onto your own computer (Lesson 1.4)
  5. Plug in tools other people own, through MCP servers (Lessons 1.6–1.7)
  6. Add memory and a subagent (Project 1)
- **What you produce:** Project 1, a personal assistant agent connected to real tools.
- **Why it comes first:** you can't diagnose an agent until you know what's inside one.
- **Hand-off to Unit 2:** the website calls this a "bad" agent on purpose: "You will have watched it fail in ways you didn't expect. Those failures are not a problem. They are the point." Keep a log of where yours breaks.

Unit 2: Diagnose a broken agent

- **Your role:** a new hire at a company that already has an agent, and it's broken.
- **The question:** what is underneath the agent, and why does it fail?
- **What changes:** industry cases start here. James: "A hobbyist will make their own agent for themselves, but in industry, if you want to make a good agent, then you have to use diagnostic tools."
- **What you learn:** diagnostic tools. James doesn't name them; the course objectives list "evaluate agents using agent observability tools."
- **What you produce:** a diagnosis of the company's agent, then a fix.
- **Seeds already planted in Unit 1** (my connections, not James's):
  - Ludwig's support bot (Lesson 1.6 slides) was a diagnosis story. The model behaved perfectly; a renamed field in another team's API made the tool return a confident wrong answer.
  - Printing which tools your agent called (Lesson 1.7) is a hand-made version of what diagnostic tools do automatically.
  - Prep 1.2 said "we are not using tracing yet," a hint that tracing comes later.

Bridge: Intent engineering

- **Where it sits:** James calls it "its own thing, kind of in between Unit 2 and Unit 3."
- **Plain meaning** (general use; the course may define it differently): stating why the agent exists and what success looks like, clearly enough that the agent can pursue the goal reliably. One common framing stacks three layers: intent (why), context (what the model knows), and prompt (how you ask).
- **Why it sits here:** Unit 2 asks why an agent fails, and Unit 3 asks what to build. Both depend on intent, because an agent is only "broken" or "working" measured against what it was meant to do.

- **Seeds already planted:** Group Day (Lesson 1.3 slides) asked “How do you measure the quality of a hamburger?” and “What constitutes a sandwich?” Both are intent questions: what counts as success, and do your users agree on it?

Unit 3: Build an agent for people who need it

- **Your role:** a new hire at a company that doesn’t have an agent. In James’s scenario it’s a nonprofit that is “not very savvy.”
- **The question:** what should we build, and for whom? James: “How do we actually make this for people rather than just making it?”
- **What you learn:**
  - **Product requirements:** finding out what people actually need before you build.
  - **Long-running agents:** agents that work across many steps or sessions.
  - **Context management:** deciding what the model sees at each step.
  - **Safety:** what could go wrong with an agent, and how to prevent it.
- **What you produce:** an agent built from scratch with a purpose. James: Unit 3 is where you “deal with the real mess” of the real world.
- **Seeds already planted:**
  - Lesson 1.7’s “too many tools” hint (every MCP server adds tools the model has to read) is context management in miniature.
  - The course objective “communicate about AI using language fitted to the audience” fits a client who isn’t tech-savvy.

Safety: deterministic vs. probabilistic

James brings up safety right after Unit 3 and frames it as “your deterministic elements versus your probabilistic”:

	Deterministic	Probabilistic
What it is	Plain code: same input, same output, every time	The model’s choices: likely, not guaranteed
An example from Unit 1	The filesystem server refuses folders you didn’t list (Lesson 1.7)	A docstring line saying “Do NOT use this tool for…” (Lesson 1.3)
Good for	Rules that must never fail	Judgment calls and language

A question to carry into Unit 3: which parts of the agent must never fail, and are those parts in deterministic code?

Capstone: Give back to the course (if time allows)

- **Timing:** James: “I don’t know this time around how much time we have; we’re going to have to play it by ear towards the end.”

- **The idea:** a project that contributes back to the course. His examples: noticing “this thing is out of date by the end of the course, let me go and fix it,” or building something like a tutor agent.
- **Your role:** contributor, building for future students.
- **Why it fits:** AI tools change fast, so a course about them goes stale fast. The Lesson 1.7 slides make the same point: “Give back when you can (a star, a bug report, a fix).”

Settled vs. still open

Settled (from James):

- The order: build, diagnose, intent, build for others, then a capstone if time allows.
- Unit 2 means using diagnostic tools on a company’s broken agent.
- Unit 3 means a nonprofit client, product requirements, long-running agents, and context management.

Still open:

- **Timing.** James plans to play the end by ear. The course repository lists Unit 1 as Weeks 1–4, Unit 2 as Weeks 5–8, and Unit 3 as Weeks 9–12; treat those as a draft.
- **Which diagnostic tools** Unit 2 uses.
- **Where safety sits:** inside Unit 3, or across Units 2 and 3.
- **“Battle of the Bots.”** The repository’s Unit 3 title is “Context Engineering + Battle of the Bots.” James didn’t mention it.
- **Whether the capstone happens.**

Getting started this week

If the repository’s four-week units hold, Unit 2 starts Monday, October 12.

1. **Confirm two dates with James:** whether Project 1 is still open, and when Unit 2 starts.
2. **Get Project 1 running, even if it’s rough.** It’s the agent you understand best, and Unit 2 builds on watching agents fail.
3. **Start a failure log.** For 3–5 moments your agent surprised you, write what you asked, what it did, what you expected, and your best guess why. This is practice for Unit 2, and Project 1’s Canvas reflection asks for the same thing.
4. **Reread Ludwig’s story** in the Lesson 1.6 slides. It’s the course’s first diagnosis and a preview of the Unit 2 mindset.
5. **Keep this roadmap next to your notes** and add one line per class under the unit you’re in.

Vocabulary for Units 2 and 3

General meanings; the course may define them more precisely.

Term	Plain meaning
Diagnostic (observability) tools	Tools that record what an agent did, step by step, so you can find where it went wrong
Tracing	A step-by-step record of one agent run: what the model saw, which tools it called, what came back
Intent engineering	Stating why the agent exists and what success looks like, clearly enough to guide it
Product requirements	What the agent must do, for whom, and how you'll know it works, gathered from the people who will use it
Context management	Choosing what the model sees at each step (instructions, tools, history, data), since its room and attention are limited
Long-running agent	An agent that works across many steps or sessions, so it must track its progress and keep its context under control
Deterministic / probabilistic	Code that behaves the same every time / model choices that can vary between runs

Sources

- James Beeson, *DS295R Course Roadmap* (voice-to-text), October 9, 2026. Primary source.
- DS295R course website: Unit 1 lessons, Project 1, course objectives. Being reorganized.
- Course repository README: planned weeks per unit.
- MindStudio, "Prompt engineering vs. context engineering vs. intent engineering": general definition of intent engineering.